An indexed PHOAS self-interpreter for PTS-rk

October 21, 2026

Abstract

The full annotated PTS-rk admits a family of typed self-interpreters. An executable external quoter assigns every typing derivation in a well-formed context a finite numerical index and normal representations of its type and term. The corresponding interpreter is itself a normal PTS-rk term, and its application reduces to the literal source term. A separate HOAS tag preserves annotated syntax injectively. The construction uses dependent type codes, explicit equality transports and a finite signature whose identity interpretation is constructed inside the original calculus.

The family depends on ranks and a numerical plan for equality repairs; uniformity over an entire rank-bounded fragment remains open. We explain the construction, compare its strong unquotation theorem with Brown–Palsberg's interpreter for F_ω, and assess whether the representation supports a continuation-passing interpretation.

1 Introduction

Brown and Palsberg's *Breaking Through the Normalization Barrier: A Self-Interpreter for F_ω*[1] shows how a strongly normalizing typed language can interpret normal representations of its own programs. Their external quoter preserves syntax, while an internal unquoter recovers the represented term by beta reduction. This combination is stronger than computing a code for the term's normal form: evaluation must be able to return the original, possibly reducible, program.

We establish this form of recovery for the full dependent PTS-rk. Given $\vdash t : T$, quotation computes finite numerical indices j and closed normal programs P, Q such that

$$\vdash I_j\, P\, Q : T, \qquad I_j\, P\, Q \longrightarrow_\beta^* t.$$

The represented type P is an ordinary argument. Each I_j is an originally typed, normal PTS-rk program. The representation uses parametric higher-order abstract syntax (PHOAS): source binders become target binders, and abstract operation parameters keep the representation normal. Supplying their identity interpretation runs the represented program. The central difficulty is dependent conversion: the types of quoted applications can disagree before those parameters are filled. We resolve these disagreements with explicit, independently realized equality fields.

The normalization bounds provide a useful computational motivation. For fixed redex-rank bound d, the checked full-PTS normalizer satisfies

$$\mathrm{cost}(\mathsf{normalize}_d(t)) \le \mathrm{tower}_{2d+1}(N + 5d + 2),$$

where N is the node count, $\mathrm{tower}_0(x) = x$ and $\mathrm{tower}_{h+1}(x) = 2^{\mathrm{tower}_h(x)}$. Every closed typed term has the computable sufficient bound $1 + \mathsf{maxRankLiteral}(t) + \mathsf{nodes}(t)$. Thus

[1] `https://doi.org/10.1145/2837614.2837623`

normalization is elementary at each fixed bound. The manuscript's non-uniform elementary representability theorem would realize each such numeric normalizer at a computable finite rank budget. That generic compiler is not yet formalized in Rocq. Moreover, numerical normalization does not itself produce an inhabitant of a represented dependent type. The interpreter constructed here supplies typed unquotation directly, without assuming that compiler.

The result is a family indexed by ranks and a finite numerical allocation plan. The plan records bounds for equality-field enumeration and can grow with the source derivation. This qualification matters both for the comparison with Brown–Palsberg and for the possible transformations of represented programs.

2 The calculus and the interpretation theorem

Terms and types share the annotated syntax

$$t ::= x \mid \mathcal{U}_i \mid \Pi x : t.t \mid \lambda x : t.t \mid t\,t.$$

We use the original PTS-rk judgments and beta reduction, including reduction inside annotations. The characteristic formation rules are

$$\Gamma \vdash \mathcal{U}_i : \mathcal{U}_0, \qquad \frac{\Gamma \vdash A : \mathcal{U}_a \quad \Gamma, x : A \vdash B : \mathcal{U}_b}{\Gamma \vdash \Pi x : A.B : \mathcal{U}_{\max(a+1,b)}}.$$

There is no cumulativity or eta conversion. Equality of source syntax below means literal equality of the original scoped, annotated terms.

An index has the form $j = (r, k, p, s)$: r is the carrier rank of the syntax tag, k bounds the finite type-code interface, p is a finite plan of numerical equality-layer budgets, and $s \le k+1$ is the result rank. For each such index the construction supplies a dependent interface

$$I_j : \Pi P : \mathsf{TyProg}_{k,p,s}.\mathsf{Pack}_r(\mathsf{TmProg}_{k,p,s}(P)) \to \mathsf{Result}_{k,p,s}(P).$$

The type program, term program and package types are defined below. They depend on the finite interface and the ordinary argument P. The source term, source type and source derivation are not parameters of I_j.

Theorem 2.1 (Full dependent indexed interpretation)**.** *For every original closed judgment $\vdash t : T$ and every tag rank r, quotation supplies finite k, p, s and closed terms P, Q such that $s \le k+1$, $P, Q, I_{r,k,p,s}$ are beta-normal, and*

$$\begin{aligned} &\vdash P : \mathsf{TyProg}_{k,p,s}, \\ &\vdash Q : \mathsf{Pack}_r(\mathsf{TmProg}_{k,p,s}(P)), \\ \vdash I_{r,k,p,s}\,P\,Q &: T, \\ I_{r,k,p,s}\,P\,Q &\longrightarrow_\beta^* t. \end{aligned}$$

Executing P with the interface's identity filling reduces to the literal source type T. For constructed faithful packages, $Q =_\beta Q'$ implies equality of their source terms, even when the derivations and numerical indices differ.

Quotation is executable on independently specified derivation data. Every original typing derivation is represented by such data; the coverage statement about original propositional typing is proved separately. The quoter does not inspect an erased Rocq proposition.

For a well-formed open context Γ, the construction first quotes the literal dependent closure $\lambda\Gamma.t$, then applies its interpreted value to the variables of Γ. This gives original typing and reduction to t in Γ. Syntax fidelity holds at a fixed source scope, including comparisons between different contexts of the same length.

2.1 Why the interpreter is a family

The *type rank* of $t : T$ is the unique a such that $T : \mathcal{U}_a$. In a well-formed context it is invariant under conversion, and application cannot increase it:

$$\mathrm{rk}_\Gamma(f\,u) \le \mathrm{rk}_\Gamma(f).$$

Indeed, if $f : \Pi x : A.B$, with $A : \mathcal{U}_a$ and $B : \mathcal{U}_b$ under the binder, the function has type rank $\max(a+1, b)$ and its application has type rank b. Hence every typed application spine headed by a fixed E has type rank at most $\mathrm{rk}(E)$.

Proposition 2.2 (Fixed-head obstruction)**.** *No one closed typed PTS-rk term E can recover every closed typed source, up to beta conversion, by a typed application spine $E\,a_1 \cdots a_m$.*

Proof. Let $A_0 = \mathcal{U}_0$, $A_{n+1} = \Pi x : A_n.\mathcal{U}_0$, $v_0 = \mathcal{U}_0$ and $v_{n+1} = \lambda x : A_n.\mathcal{U}_0$. The original rules give $A_n : \mathcal{U}_n$ and $v_n : A_n$. If E has type rank R, no typed application spine of E can be convertible to v_{R+1}. □

The argument allows arbitrary type and code arguments, including an internal numeral purporting to select a rank. It concerns this precise application interface; externally generated abstractions around the result fall outside it. A family of different heads I_j avoids the obstruction.

Type rank must also be distinguished from the redex and literal bounds used in quotation. For example, $\mathcal{U}_m : \mathcal{U}_0$ has type rank zero for every m, while its representation must retain the literal m. The interface budget therefore includes source literals and the ranks needed throughout the derivation.

3 Representation and evaluation

The construction separates three jobs. A finite Tarski signature represents dependent types and their inhabitants. Checked equality fields repair the mismatches introduced by quotation. A raw HOAS tag records source syntax independently of those repairs. Closing over the signature produces the normal arguments of the interpreter.

3.1 Dependent codes and their identity interpretation

For each required rank a, the signature contains a code carrier and a decoder,

$$C_a : \mathcal{U}_0, \qquad \mathsf{El}_a : C_a \to \mathcal{U}_a.$$

For $c = \max(a+1, b)$ it also contains a dependent product constructor

$$p_{a,b} : \Pi A : C_a.(\mathsf{El}_a A \to C_b) \to C_c.$$

Thus a code family $B : \mathsf{El}_a A \to C_b$ binds an actual target variable of the decoded domain type. This is the dependent HOAS component of the representation.

The decoded product code need not be definitionally equal to the product of the decoded constituents. The signature therefore includes a witness relating

$$X = \Pi x : \mathsf{El}_a A.\mathsf{El}_b(Bx), \qquad Y = \mathsf{El}_c(p_{a,b}\, A\, B).$$

Equality is represented by ordinary PTS syntax. For $D : \mathcal{U}_a$ and $u, v : D$, use the rank-indexed Leibniz type

$$\mathsf{Eq}_d(D, u, v) = \Pi M : (D \to \mathcal{U}_d).M\, u \to M\, v, \qquad \mathsf{refl}_d(D, u) = \lambda M : (D \to \mathcal{U}_d).\lambda z : M\, u.z.$$

Its sort is $\mathcal{U}_{\max(a+2,d+1)}$. The product-law field has type

$$\ell_{a,b} : \Pi A : C_a.\Pi B : (\mathsf{El}_a A \to C_b).\mathsf{Eq}_{c+1}(\mathcal{U}_c, X, Y).$$

A witness e at this equality type gives casts in both directions:

$$\begin{aligned} \mathsf{cast}^{+}(e) &= e\,(\lambda Z : \mathcal{U}_c.X \to Z)\,(\lambda x : X.x) : X \to Y, \\ \mathsf{cast}^{-}(e) &= e\,(\lambda Z : \mathcal{U}_c.Z \to X)\,(\lambda x : X.x) : Y \to X. \end{aligned}$$

The predicate rank $c + 1$ is sufficient for both arrow motives. Applying these casts to a function or an encoded function gives the typed abstraction and application operations. Universe codes $u_a : C_0$ have the analogous law $\mathsf{Eq}_1(\mathcal{U}_0, \mathsf{El}_0(u_a), C_a)$, connecting a decoded universe with the carrier of its type codes.

All fields have concrete identity interpretations:

$$\begin{aligned} C_a &= \mathcal{U}_a, & \mathsf{El}_a &= \lambda A : \mathcal{U}_a.A, \quad u_a = \mathcal{U}_a, \\ p_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : (A \to \mathcal{U}_b).\Pi x : A.Bx. \end{aligned}$$

The laws specialize to reflexivity at beta-convertible endpoints. Their inhabitants are constructed and typed in the original PTS. At these inhabitants the casts reduce to identity functions, so abstraction and application recover the literal function and its application.

For a budget k, carriers through $k + 1$ suffice for product operations with $a, b \le k$. This gives a finite, well-formed signature with normal declarations and a normal, originally typed identity filling. The same properties will hold after the equality extensions used by quotation.

3.2 Equality repairs under binders

Neutral casts keep representations normal, but their round trips do not cancel by beta conversion before the identity filling is supplied. In a dependent application, such a change of argument can also change its result type. The quoter therefore needs more than the basic product law.

It extends an already realized signature $\mathcal{S}$ with fields of the form

$$e : \Pi\Delta.\mathsf{Eq}_d(D, u, v),$$

where Δ is a local dependent telescope. A field is accepted only when its components have the required original typings, its declaration is normal, and its endpoints become beta-convertible after the existing identity filling ι. These are decidable checks on ordinary annotated syntax. They do not assume that a source term has a quotation.

The common normal form of the filled endpoints supplies a typed reflexivity realizer. Closing over Δ and normalizing this realizer gives a normal field value whose application to the telescope arguments, a motive and a payload reduces to that literal payload. Normalization here constructs the equality witness; it does not evaluate the represented source program.

Each layer enumerates all accepted fields within specified bounds on telescope length, syntax nodes and sort literals. A finite plan p is a sequence of such numerical budgets; later layers refer only to earlier ones. The quoter computes a sufficient request whenever it needs an alignment. This makes the signature and its filling depend only on (k, p), while allowing the plan itself to grow with the derivation. No source expression or semantic source value is stored in the external index.

3.3 Closing programs and retaining syntax

Let $\mathcal{S} = \mathcal{S}_{k,p}$ be the completed signature and ι its identity filling. The notation $\Pi\mathcal{S}$ means dependent quantification over its fields, and $P\,\mathcal{S}$ means application to those bound variables. Define

$$\begin{aligned}
\mathsf{TyProg}_{k,p,s} &= \Pi\mathcal{S}.C_s,\\
\mathsf{TmProg}_{k,p,s}(P) &= \Pi\mathcal{S}.\mathsf{El}_s(P\,\mathcal{S}),\\
\mathsf{Result}_{k,p,s}(P) &= \mathsf{El}_s[\iota]\,(P\,\iota).
\end{aligned}$$

If quotation constructs a type code $T^q : C_s$ and a term $t^q : \mathsf{El}_s(T^q)$ under $\mathcal{S}$, their closures are

$$P = \lambda\mathcal{S}.T^q, \qquad q = \lambda\mathcal{S}.t^q.$$

The basic evaluator $\lambda P.\lambda q.q\,\iota$ is directly typed at the dependent result interface. Since the filling interprets El_s as identity, recovery of T^q and t^q gives $P\,\iota \longrightarrow_\beta^* T$ and $q\,\iota \longrightarrow_\beta^* t$.

To retain literal syntax, we also use a homogeneous raw HOAS code $\langle t\rangle_r : \mathsf{Tag}_r$. This code abstracts over a carrier $X : \mathcal{U}_r$ and handlers

$$z : X, \quad s, u : X \to X, \quad p, l : X \to (X \to X) \to X, \quad a : X \to X \to X.$$

For a variable environment ρ, its body is the structural fold

$$\begin{aligned} F_\rho(\mathcal{U}_m) &= u(s^m z), & F_\rho(x) &= \rho(x), \\ F_\rho(\Pi x : A.B) &= p(F_\rho(A))(\lambda x : X.F_{\rho,x}(B)), \\ F_\rho(\lambda x : A.b) &= l(F_\rho(A))(\lambda x : X.F_{\rho,x}(b)), \\ F_\rho(f\, v) &= a(F_\rho(f))(F_\rho(v)). \end{aligned}$$

Closing over the carrier and handlers gives the tag; open raw syntax is closed over its carrier variables as well. The tag type has sort $\mathcal{U}_{r+3}$. Every constructed tag is normal, and structural inversion recovers the exact source tree, including annotations. This tag represents raw syntax; the dependent program supplies its typed interpretation.

For a program type V, set

$$\begin{aligned} \mathsf{Pack}_r(V) &= (V \to \mathsf{Tag}_r \to V) \to V, \\ \mathsf{pack}_r(V, q, t) &= \lambda h : (V \to \mathsf{Tag}_r \to V).h\, q\, \langle t \rangle_r. \end{aligned}$$

The faithful evaluator selects q from this package and runs it with ι. Suppressing (k, p, s) on the interface types, it is

$$\begin{aligned} I_j = \lambda P : \mathsf{TyProg}.\lambda Q : \mathsf{Pack}_r(\mathsf{TmProg}(P)). \\ \big(Q\, (\lambda q : \mathsf{TmProg}(P).\lambda z : \mathsf{Tag}_r.q)\big)\, \iota. \end{aligned}$$

Its selected payload is the PHOAS program itself. Quotation does not have to store the source's normal value.

One annotation detail is essential for full normality. The package uses the normal type $V = \Pi\mathcal{S}.\mathsf{El}_s(T^q)$ in its binder. Original conversion then assigns it the interface type $\mathsf{Pack}_r(\mathsf{TmProg}(P))$. Writing the reducible expression $P\,\mathcal{S}$ directly into that binder would spoil annotation normality. All resulting package and interpreter typings are derived before using their reduction equations.

4 Constructing quotation

Quotation proceeds structurally through the six original typing rules. Its state consists of the realized signature, the translated local context and a typed variable environment. Each result records normal type and term codes, original typings, and literal recovery after filling. The source context and all its annotations remain available throughout.

A finite constructor view records the shape of the source type separately from its current target code. This separation is needed because a conversion repair can change the code at the root without changing the source's product structure. In particular, a product view retains its domain and codomain children, so dependent substitution can operate on their finite syntax.

The cases are as follows.

- **Sorts and variables.** Use the corresponding universe code or the variable environment, with the universe casts where needed.

- **Products and abstractions.** Quote the domain, extend the local context by its normal decoded code, and quote the given body premise. Source binding is represented by target binding. The local operations contract administrative redexes in generated annotations and preserve fresh variables literally.
- **Application.** Inspect the function type's product view. Two checked equality layers align the argument's type code and the function's code with the required product code. Apply the typed operation, then substitute the quoted argument structurally into the codomain view.
- **Conversion.** Quote the rule's supplied target-formation premise and align the payload with that independently constructed code. The new field erases to identity under the filling.

The numerical budget bounds the ranks and literals needed in every recursive premise. Signature growth preserves the local context and variable environment. Substitution of the normal neutral argument representations preserves full normality, including annotations.

The constructor views also make termination explicit. Application substitutes into an existing finite view; it does not recursively quote a newly synthesized derivation of the substituted type. Similarly, conversion uses its existing formation premise. A nonrecursive repair handles a term viewed as a type at a literal universe. Thus the sort cycle $\mathcal{U}_0 : \mathcal{U}_0$ creates no cycle in quotation.

Proof of the interpretation theorem. Induct on the independent source derivation, maintaining the typing, normality and recovery invariants above. Each requested equality layer exists by finite enumeration and is realized by the already proved endpoint agreement. This constructs the finite plan and the open quotation judgment.

Close the type and term codes over the final signature, and package the term program with its raw HOAS tag. The signature and filling are normal, so the resulting P, Q, I_j are normal. Original typed substitution gives the evaluator's application typing. Selection and signature instantiation reduce it to $t^q[\iota]$, which reduces to the literal t; the type program similarly recovers T.

Equality of faithful packages implies equality of their tags, hence of their annotated sources. For beta-convertible packages, normality and confluence first give literal package equality. Finally, coverage of the independent derivation datatype establishes the theorem for every original typing judgment. □

5 Comparison with Brown–Palsberg

Our result is a **Brown–Palsberg-style indexed strong unquoter**. The qualification identifies the main difference: their interpreter is one polymorphic term, while ours is an externally generated family. There is also a difference in the operations established on the representations.

5.1 Recovery, fidelity and dependence

Brown–Palsberg's Theorem 7.4 has the form

$$\mathsf{unquote}\ulcorner T\urcorner\ulcorner t\urcorner \longrightarrow_\beta^* t, \qquad \mathsf{unquote} : \forall\alpha : U.\mathsf{Exp}\,\alpha \to \mathsf{Op}\,\mathsf{Id}\,\alpha.$$

Here U is their universe of represented types. The same fixed $\mathsf{unquote}$ accepts each represented type as an argument. Our P plays the corresponding role, and both constructions use external quotation and recover the unreduced source. Our ordinary type argument therefore matches a central part of their interface. The varying (k, p, s), however, changes the interpreter term itself. The fixed-head obstruction proves that a direct all-rank copy of their application interface is unavailable in PTS-rk. Whether our plan can be replaced by one rank index for each whole bounded fragment remains open.

The positive theorem covers dependent products, conversion in types and all original PTS-rk universes. Its normality includes annotations. Brown–Palsberg's Figure 2 term reduction does not reduce inside type annotations, and their Definition 6.1 and Theorem 6.3 formulate fidelity up to beta-equivalence of types. Our tag retains literal annotation syntax, so even distinct convertible source terms remain distinguishable. This stronger retention property does not imply that two derivations of the same source produce identical quotations; canonical derivation-independence is a separate property.

Their experiments apply each operation to its own representation using the same fixed operation. Our coverage theorem gives self-interpretation across potentially different indices. Both results support self-reference, but the same-index claim is stronger than what we prove here.

5.2 Interpretation as a reusable fold

Their deep Church encoding abstracts over a type operator $F : * \to *$ and four handlers for term abstraction, term application, type abstraction and type application. Replacing these parameters through $\mathsf{foldExp}$ yields unquotation, an abstraction test, size, a normal-form test and CPS. This ability to change interpretation is their distinction between deep and shallow representation.

Our dependent program similarly abstracts over an interface, but that interface also contains laws. An alternative interpretation must supply originally typed inhabitants for every product, universe and checked equality field. The completed theorem constructs its identity filling. The raw tag admits separate syntax folds; this does not yet supply the paper's suite of interpretations of the full dependent program.

Consequently, the construction meets the strong-unquotation part of the Brown–Palsberg approach, with a less uniform interface and richer source dependence. Calling it their full deep interpreter would also require the additional interpretations. The next section examines CPS as a concrete test.

Brown–Palsberg support their mathematical proofs with an implementation and type-checked experiments. Our quotation and interpretation theorems are kernel-checked in Rocq. The exhaustive equality enumeration has no practical efficiency bound, and the extracted sort-quotation smoke test establishes only that leaf execution path. Proof coverage and practical compiler performance remain distinct claims.

6 Can the representation support CPS?

Brown–Palsberg's Figure 4 interprets representations under the answer-type-polymorphic continuation operator

$$\mathsf{Ct}\, A = \forall R : *.(A \to R) \to R, \qquad \mathsf{cps} : \forall \alpha : U.\mathsf{Exp}\, \alpha \to \mathsf{Op}\, \mathsf{Ct}\, \alpha.$$

Their displayed transformation is call-by-name; a call-by-value variant is also reported. The result is a CPS program, not necessarily a fresh syntax code. They implement, type-check and test CPS, but explicitly leave its semantic correctness proof open.

For our representation, continuation types are expressible at larger ranks, while a full dependent CPS interpretation remains open. The calculations in this section are mathematical feasibility arguments, not additional Rocq CPS theorems.

6.1 Rank budgets and a simply typed test

For $A : \mathcal{U}_a$ and $R : \mathcal{U}_b$, the original product rule gives

$$\mathsf{Cont}_R(A) = (A \to R) \to R : \mathcal{U}_{h(a,b)}, \qquad h(a,b) = \max(a+2, b+1).$$

The answer-polymorphic version at external rank b has the same rank:

$$\mathsf{Ct}_b(A) = \Pi R : \mathcal{U}_b.(A \to R) \to R : \mathcal{U}_{h(a,b)}.$$

The outer binder contributes only rank 1, since $\mathcal{U}_b : \mathcal{U}_0$. As $h(a,b) > a$, this operator cannot simply replace a family $\mathcal{U}_a \to \mathcal{U}_a$. Rank-indexed interfaces must account for its growth. Instantiating the answer binder with A also requires the matching judgment $A : \mathcal{U}_b$; there is no cumulative inclusion between ranks.

For a concrete simply typed test, fix $O : \mathcal{U}_a$ and $R : \mathcal{U}_b$, and set

$$V(o) = O, \qquad C(A) = \mathsf{Cont}_R(V(A)), \qquad V(A \to B) = C(A) \to C(B).$$

A call-by-name translation, with a source variable $x : A$ represented by $x : C(A)$, is

$$\begin{aligned} \mathcal{C}(x) &= \lambda k : V(A) \to R.\, x\, k, \\ \mathcal{C}(\lambda x : A.t) &= \lambda k : V(A \to B) \to R.\, k\, (\lambda x : C(A).\mathcal{C}(t)), \\ \mathcal{C}(f\, u) &= \lambda k : V(B) \to R.\, \mathcal{C}(f)\, (\lambda g : C(A) \to C(B).g\, \mathcal{C}(u)\, k). \end{aligned}$$

A structural typing induction assigns $\mathcal{C}(t)$ the type $C(A)$. The variable clause introduces its lambda explicitly, using no eta rule. Writing v, c for the value and computation ranks gives

$$v(o) = a, \quad c(A) = \max(v(A)+2, b+1), \quad v(A \to B) = \max(c(A)+1, c(B)).$$

These are finite, effective budgets, and the clauses have the form of HOAS handlers. Even here the budget can depend on type structure: for $a = b = 0$, $A_0 = o$ and $A_{n+1} = o \to A_n$, every A_n with $n \geq 1$ has source rank 1, whereas $c(A_n) = 2n + 3$. This limits the displayed translation's rank uniformity; it is not a lower bound for every possible encoding.

6.2 The dependent laws are the main obstacle

A filling ρ of the complete signature would give a typed interpretation $q\,\rho$. The naive continuation replacement fails to supply such a filling for three connected reasons.

First, its larger output ranks disagree with the declared decoder sorts. Second, decoding a product code as a continuation-wrapped function changes the right endpoint of the product

law, while the left endpoint is still the ordinary dependent product of decoded constituents. The identity interpretation's reflexivity witness then ceases to type check. Non-convertibility alone does not prove that the internal equality type is uninhabited; a different realization would need a separate proof. Third, the extra equality fields are checked after identity filling. Their validity under a continuation filling has not been established.

The underlying dependent issue is that $f : \Pi x : A.B(x)$ applied to $u : A$ has type $B(u)$. Translating u into a computation changes an argument of a type family as well as evaluation order. An interpretation must therefore preserve the needed substitution and conversion equations, or realize their repairs internally.

For example, naive call-by-value CPS does not preserve all source beta conversion. In the context $g : O \to O$, $v : O$, $y : O$,

$$(\lambda x : O.y)\,(g\,v) \longrightarrow_\beta^* y.$$

With translated variable $\widehat{g} : O \to \mathsf{Cont}_R(O)$, the two standard fixed-answer translations reduce administratively to

$$\lambda k : O \to R.\widehat{g}\,v\,(\lambda z : O.k\,y), \qquad \lambda k : O \to R.k\,y.$$

For normal annotations these are distinct beta-normal terms. Dependent source types may use the original unrestricted conversion, so this recipe cannot preserve it by beta conversion alone. The example does not rule out call-by-name CPS or a translation with explicit dependent transports.

Existing positive dependent-CPS theorems require care here. Barthe, Hatcliff and Sørensen (1999)[2] treat annotated pure type systems, but their logical-sort condition requires $(\mathsf{Prop}, \mathsf{Prop}, \mathsf{Prop})$ as a product rule. PTS-rk has no such sort, since $\max(a+1, a) = a+1$. Bowman, Cong, Rioux and Ahmed (2018)[3] prove dependent CPS translations using answer-type polymorphism and additional target typing and equivalence rules. Those rules are not available in our beta-only calculus. Neither result directly supplies the required signature filling.

6.3 What would establish feasibility

An external derivation-directed CPS translation could produce an original target derivation and then use our quoter to obtain a normal faithful code at a new index. An internal CPS fold requires more: it must operate on P, Q without access to the external derivation or its constructor-view metadata, and supply a typed continuation interpretation of the laws.

A useful first target is a dependent continuation product/application handler together with an actual inhabitant for one nontrivial contextual equality field. A redesigned representation might retain a direct value for dependent type indices alongside a continuation computation, provided their relationship is proved under substitution and binders. Simply wrapping evaluation in $\lambda k.k\,(I_j\,P\,Q)$ does not transform the represented applications.

Thus higher ranks and HOAS permit a concrete simply typed CPS construction, but the present full dependent interface has no established CPS filling. The outstanding question is dependent coherence, not a general impossibility of continuations in PTS-rk.

[2] `https://www.cs.ru.nl/~freek/courses/tt-2017/papers/10.1023-A-1010000206149.pdf`

[3] `https://www.williamjbowman.com/resources/cps-sigma.pdf`

7 Formalization and remaining questions

The construction is in `PTSrk.SelfInterpreter` and uses the original `Syntax.Term`, `Typing.typ` and `Ranks.RankSorts`. Its main entry point is `FullDependentInterpreter`: `closed_quote` is executable quotation; `every_closed_original` and `every_original` give closed and open coverage; `closed_type_recovers` gives literal type recovery. The corresponding injectivity theorems retain annotated source syntax. `RecursiveQuotation` implements the six-rule induction. The development README gives the module map and verification commands.

The interpreter theorem has fresh closed assumption reports and recursive kernel checks against pristine upstream. No new axiom, admission, unsafe cast or altered typing rule is used. The CPS analysis above is outside that formalized theorem.

The remaining questions concern a rank-only interface, a smaller and efficiently generated collection of equality laws, and a typed dependent CPS interpretation. These strengthen the completed finite-plan self-interpreter and its strong recovery theorem.